

Arceos驱动测试技术文档

Arceos驱动测试技术文档

一、测试概述

- 1.1 测试背景与目标
- 1.2 测试环境构成
- 1.3 硬件连接规范

二、基本驱动的测试

- 2.1 测试前的准备
- 2.2 测试机上发起测试
 - 2.2.1 reset测试
 - 2.2.2 clock测试
 - 2.2.3 GPIO测试
 - 2.2.4 I2C测试
 - 2.2.5 pinmux测试
 - 2.2.6 PWM测试
 - 2.2.7 spi测试
 - 2.2.8 timer测试
 - 2.2.9 UART测试
 - 2.2.10 watchdog测试

三、对复杂驱动的测试

- 3.1 测试前的准备
- 3.2 DMA和GIC测试
 - 3.2.1 测试原理与目的
 - 3.2.2 执行测试命令
 - 3.2.3 测试结果验证
- 3.3 UVC和USB测试
 - 3.3.1 测试原理与目的
 - 3.3.2 硬件准备
 - 3.3.3 执行测试命令
 - 3.3.4 测试结果验证
- 3.4 PCIE和IGB测试
 - 3.4.1 测试原理与目的
 - 3.4.2 硬件准备
 - 3.4.3 执行测试命令
 - 3.4.4 测试结果验证
- 3.5 FXMAC测试
 - 3.5.1 测试原理与目的
 - 3.5.2 执行测试命令
 - 3.5.3 测试结果验证

四、测试总结

一、测试概述

1.1 测试背景与目标

在嵌入式系统开发中，驱动程序作为硬件与操作系统之间的桥梁，其稳定性和功能性直接决定了整个系统的运行效率与可靠性。Arceos 作为一款面向嵌入式场景的操作系统，需在不同硬件平台上实现对各类外设的有效驱动支持。飞腾派开发板基于国产飞腾处理器，具备高性能、低功耗等特性，是 Arceos 操作系统重要的适配硬件之一。

本次测试旨在全面验证 Arceos 操作系统在飞腾派开发板上各类驱动的功能正确性、稳定性及兼容性，覆盖基本外设驱动与复杂功能驱动，为后续 Arceos 系统在飞腾派平台的应用与优化提供可靠的测试依据，确保驱动程序能够满足实际嵌入式应用场景的需求。

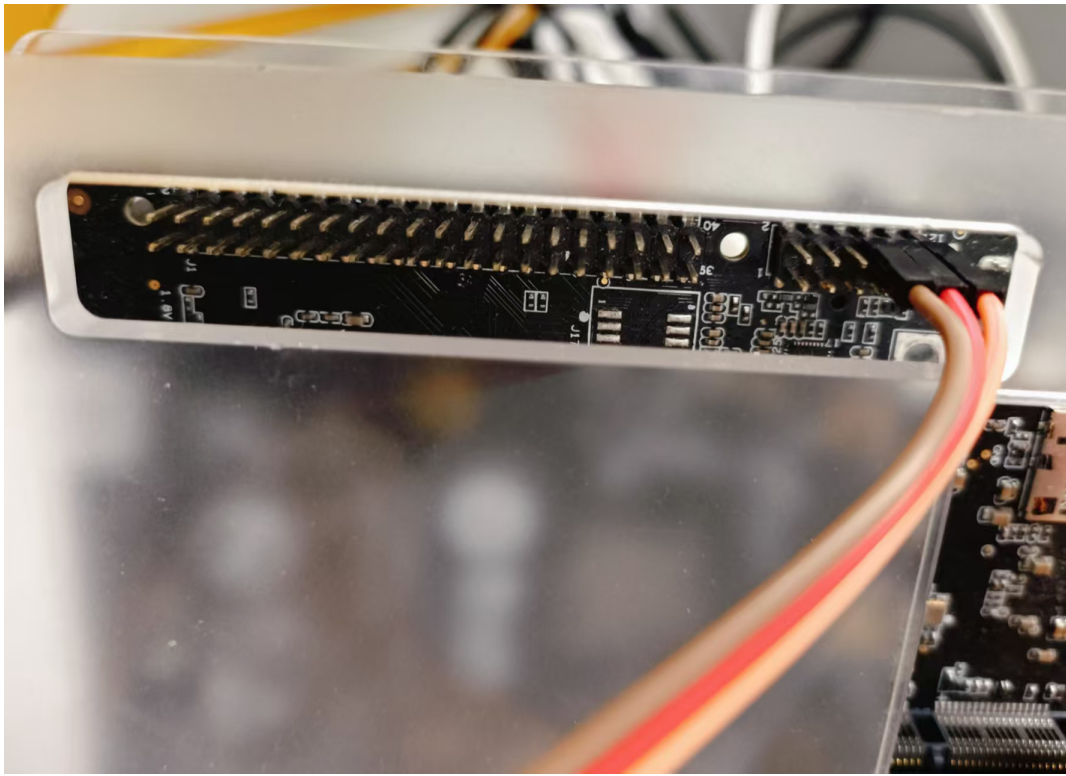
1.2 测试环境构成

整个测试环境由两个部分组成，一个作为测试机，其上运行Linux，另一个是飞腾派（被测试开发板），其上运行Arceos。具体配置与连接方式如下：

设备	配置说明	功能角色
测试机	操作系统：Linux（推荐 Ubuntu 24.04 及以上版本） 硬件要求：CPU 双核及以上，内存 4GB 及以上，空闲磁盘空间 10GB 及以上	1. 编译测试所需镜像文件 2. 安装并运行测试依赖工具与脚本 3. 发起驱动测试任务，控制测试流程 4. 接收并分析测试结果，生成测试日志
飞腾派开发板	处理器：飞腾系列嵌入式 CPU 内存：2GB 及以上 DDR4 存储：支持 microSD 卡或 eMMC 存储 接口：12 针调试串口、40针通用 GPIO 接口、USB 接口等	1. 运行 Arceos 操作系统，作为驱动测试的目标硬件平台 2. 提供各类外设接口（如 UART、I2C、SPI 等），供驱动测试调用 3. 通过调试串口与测试机通信，反馈测试过程中的硬件状态

1.3 硬件连接规范

测试机需通过 **USB 转 TTL 模块** 与飞腾派的 12 针调试串口连接，实现数据交互与测试控制，具体接线对应关系如下：



- 飞腾派12针接口：12pin为GND，接USB转TTL的GND
- 飞腾派12针接口：10pin为RX，接USB转TTL的TX
- 飞腾派12针接口：8pin为TX，接USB转TTL的RX

连接完成后，可通过测试机的 `ls /dev/ttyUSB*` 命令确认串口设备是否正常识别。

二、基本驱动测试

基础驱动测试覆盖飞腾派开发板上常用的外设驱动，包括 reset、clock、GPIO 等，测试流程统一分为“测试机准备”“飞腾派准备”“发起测试”三个阶段，各驱动测试的核心差异体现在测试内容、专属配置与结果验证方式上。

2.1 测试前的准备

- 测试机上的准备

```
# 在测试机上运行
git clone --recursive https://github.com/shzhxh/driver-test.git
cd driver-test
make build # 编译测试镜像，生成的镜像在shell目录下

# 安装测试依赖
python3 -m venv ~/.venv
source ~/.venv/bin/activate
pip3 install -r ./scripts/requests.txt
deactivate
```

- 飞腾派上的准备

```
# 在测试机上运行。要求：飞腾派上已运行Linux，且与测试机在同一局域网。
scp shell/shell*.bin user@192.168.1.100:arceos.bin

# 在飞腾派上运行
sudo shutdown -r now # 重启飞腾派，按任意键进入uboot
# 在uboot下执行如下命令
ext4load mmc 0:1 0x90100000 /home/user/arceos.bin
dcache flush
go 0x90100000 # 启动Arceos
```

2.2 测试机上发起测试

1. 关闭串口占用程序。当开始测试时，测试机上读写飞腾派调试串口的程序应保持关闭。
2. 要修改的python文件里的配置，使与实际情况相符。
 - `scripts/conftest.py`：`port` 默认是 `/dev/ttyUSB0`。若测试机识别的串口为其他设备（如 `/dev/ttyUSB1`），需修改 `port` 参数。
 - `scripts/test_uart_driver.py`：配置 UART 测试专用串口，默认值为 `/dev/ttyUSB1`，需根据实际连接的 USB 转 TTL 模块端口调整 `TEST_UART_PORT` 参数。

2.2.1 reset测试

Watchdog（看门狗）驱动是嵌入式系统中保障稳定性的关键组件，其核心功能是：若系统出现死锁或无响应，Watchdog 定时器超时后会自动触发系统复位（reset）。

本次 reset 测试的目的：验证 Arceos 的 Watchdog 驱动能否正常初始化、启动定时器，并在“不喂狗”（未按时重置定时器）的情况下触发飞腾派重启，确保驱动的故障恢复能力。

1. 前置检查

确认飞腾派已成功启动 Arceos，且测试机与飞腾派的串口通信正常。

2. 执行测试命令

在测试机的 Python 虚拟环境中，通过 `pytest` 执行 reset 测试（仅运行标记为 `reset` 的测试用例）：

```
# 运行测试。
source ~/.venv/bin/activate
pytest -v -m reset # 日志记录在output目录下
deactivate
```

3. 测试结果验证

观察测试机终端输出与飞腾派状态。飞腾派应在测试过程中自动重启。

```
szx@5600g:driver-test$ scp shell/shell_aarch64-phytium-pi.bin user@172.16.203.132:arceos.bin
user@172.16.203.132's password:
shell_aarch64-phytium-pi.bin                                100% 320KB  5.2MB/s  00:00
szx@5600g:driver-test$ pytest -v -m reset
===== test session starts =====
platform linux -- Python 3.12.3, pytest-7.4.4, pluggy-1.4.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /home/szx/Code/arceos-org/driver-test
configfile: pytest.ini
collected 32 items / 30 deselected / 2 selected

scripts/test_cru_driver.py::test_cru_initialization PASSED [ 50%]
scripts/test_watchdog_driver.py::test_watchdog_functionality PASSED [100%]

===== 2 passed, 30 deselected in 0.24s =====
```

2.2.2 clock测试

Clock（时钟）驱动是所有外设工作的基础，外设（如 UART、SPI）的速率、定时器精度均依赖于稳定的时钟信号。

本次 clock 测试的目的：验证 Arceos 的 Clock 驱动能否正常初始化 CRU 模块，并将指定外设的时钟频率精确设置为 50MHz，确保时钟信号的稳定性与准确性。

1. 执行测试命令

在测试机虚拟环境中运行 clock 测试：

```
source ~/.venv/bin/activate
pytest -v -m clock # 日志记录在output目录下
deactivate
```

2. 测试结果验证

```
szx@5600g:driver-test$ pytest -v -m clock
===== test session starts =====
platform linux -- Python 3.12.3, pytest-7.4.4, pluggy-1.4.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /home/szx/Code/arceos-org/driver-test
configfile: pytest.ini
collected 32 items / 30 deselected / 2 selected

scripts/test_clock_driver.py::test_clock_initialization PASSED [ 50%]
scripts/test_clock_driver.py::test_clock_functionality PASSED [100%]

===== 2 passed, 30 deselected in 0.74s =====
```

2.2.3 GPIO测试

GPIO（通用输入输出）驱动是嵌入式系统中控制外部硬件（如指示灯、继电器）的核心，支持配置引脚为输入或输出模式，并通过设置电平（高/低）实现对外部设备的控制。

本次 GPIO 测试的目的：验证 Arceos 的 GPIO 驱动能否正常初始化指定引脚、配置为输出模式，并通过切换高低电平控制调试灯的亮灭，确保驱动的电平控制功能正常。

1. 执行测试命令

```
source ~/.venv/bin/activate
pytest -vs -m gpio # 日志记录在output目录下
deactivate
```

2. 测试结果验证

测试过程中，测试机会提示“即将执行 gpio_test 测试命令，请观察调试灯，按 Enter 键开始...”，按下 Enter 后，LED 应交替闪烁（高电平时亮，低电平时灭），周期约 1s。观察到 LED 闪烁后，在测试机终端输入“OK”并回车，测试日志应显示 `PASSED`。

```
szx@5600g:driver-test$ pytest -vs -m gpio
===== test session starts =====
platform linux -- Python 3.12.3, pytest-7.4.4, pluggy-1.4.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /home/szx/Code/arceos-org/driver-test
configfile: pytest.ini
collected 32 items / 30 deselected / 2 selected

scripts/test_gpio_driver.py::test_gpio_initialization PASSED
scripts/test_gpio_driver.py::test_gpio_functionality 即将执行 gpio_test 测试命令，请观察调试灯
按Enter键开始...
如观察到调试灯闪烁，请输入OK，否则直接按entry键：OK
PASSED
===== 2 passed, 30 deselected in 17.10s =====
```


2.2.4 I2C测试

I2C（Inter-Integrated Circuit）是一种双线串行通信协议，广泛用于连接传感器、显示屏等外设。Arceos 的 I2C 驱动需实现：初始化 I2C 控制器、发送/接收数据、与外设建立通信等功能。

本次 I2C 测试的目的：通过向 I2C OLED 屏写入显示数据，验证 I2C 驱动能否正常初始化、与外设建立连接，并正确传输数据，确保驱动的通信功能正常。

1. 硬件准备

需要准备一个I2C oled屏，与飞腾派的40pin接口连接，接线方式如下：



- 飞腾派40pin接口：1pin为3V3，接OLED屏的VCC

- 飞腾派40pin接口：9pin为GND，接OLED屏的GND
- 飞腾派40pin接口：3pin为SDA，接OLED屏的SDA
- 飞腾派40pin接口：5pin为SCL，接OLED屏的SCL

2. 执行测试命令

```
source ~/.venv/bin/activate
pytest -vs -m i2c # 日志记录在output目录下
deactivate
```

3. 测试结果验证

I2C屏幕测试结果：



I2C脚本运行结果：

```
szx@5600g:driver-test$ pytest -vs -m i2c
===== test session starts =====
platform linux -- Python 3.12.3, pytest-7.4.4, pluggy-1.4.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /home/szx/Code/arceos-org/driver-test
configfile: pytest.ini
collected 32 items / 30 deselected / 2 selected

scripts/test_i2c_driver.py::test_i2c_initialization PASSED
scripts/test_i2c_driver.py::test_i2c_functional 即将执行 i2c_test 测试命令，请观察I2C显示屏，按
Enter键开始...
如观察到I2C显示屏亮起，请输入OK，否则直接按entry键：OK
PASSED
===== 2 passed, 30 deselected in 6.01s =====
```

2.2.5 pinmux测试

pinmux（引脚复用）是嵌入式硬件中实现引脚多功能的关键技术：同一引脚可通过 pinmux 配置为不同功能（如 GPIO、UART_TX、SPI_SCK 等）。Arceos 的 pinmux 驱动需实现：读取引脚当前功能、配置引脚为目标功能、验证功能切换有效性等。

本次 pinmux 测试的目的：将飞腾派的指定引脚从默认的 GPIO 功能配置为 UART 功能，验证 pinmux 驱动能否正常初始化、切换引脚功能，并确保切换后 UART 驱动可正常使用（后续 UART 测试会依赖此配置）。

1. 执行测试命令

```
source ~/.venv/bin/activate
pytest -v -m pinmux # 日志记录在output目录下
deactivate
```

2. 测试结果验证

```
szx@5600g:driver-test$ pytest -vs -m pinmux
===== test session starts =====
platform linux -- Python 3.12.3, pytest-7.4.4, pluggy-1.4.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /home/szx/Code/arceos-org/driver-test
configfile: pytest.ini
collected 32 items / 30 deselected / 2 selected

scripts/test_pinmux_driver.py::TestPinmuxDriver::test_pinmux_initialization PASSED
scripts/test_pinmux_driver.py::TestPinmuxDriver::test_pinmux_functionality PASSED

===== 2 passed, 30 deselected in 0.23s =====
```

2.2.6 PWM测试

PWM（脉冲宽度调制）是通过改变脉冲信号的占空比（高电平时间/周期）来控制外设的技术，广泛用于电机调速、LED 亮度调节等场景。Arceos 的 PWM 驱动需实现：初始化 PWM 控制器、设置频率与占空比、输出 PWM 信号等功能。

本次 PWM 测试的目的：验证 PWM 驱动能否正常初始化，并输出不同占空比（1%、25%、50%、75%、100%）的 PWM 信号，确保驱动对占空比的精确控制能力。

1. 执行测试命令

```
source ~/.venv/bin/activate
pytest -v -m pwm # 日志记录在output目录下
deactivate
```

2. 测试结果验证

测试日志应显示 7 个测试用例全部 PASSED。

```
szx@5600g:driver-test$ pytest -v -m pwm
===== test session starts =====
platform linux -- Python 3.12.3, pytest-7.4.4, pluggy-1.4.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /home/szx/Code/arceos-org/driver-test
configfile: pytest.ini
collected 32 items / 25 deselected / 7 selected

scripts/test_pwm_driver.py::test_pwm_initialization PASSED [ 14%]
scripts/test_pwm_driver.py::test_pwm_functional PASSED [ 28%]
scripts/test_pwm_driver.py::test_pwm_duty_cycle_variations[1] PASSED [ 42%]
scripts/test_pwm_driver.py::test_pwm_duty_cycle_variations[25] PASSED [ 57%]
scripts/test_pwm_driver.py::test_pwm_duty_cycle_variations[50] PASSED [ 71%]
scripts/test_pwm_driver.py::test_pwm_duty_cycle_variations[75] PASSED [ 85%]
scripts/test_pwm_driver.py::test_pwm_duty_cycle_variations[100] PASSED [100%]

===== 7 passed, 25 deselected in 0.29s =====
```

2.2.7 spi测试

SPI（Serial Peripheral Interface）是一种高速同步串行通信协议，支持全双工通信，常用于连接 Flash、ADC 等高速外设。Arceos 的 SPI 驱动需实现：初始化 SPI 控制器、配置通信参数（如时钟极性、相位）、发送/接收数据等功能。

本次 SPI 测试采用“自收自发”模式：将飞腾派 SPI 接口的 TX（发送）引脚与 RX（接收）引脚短接，驱动发送一组测试数据后，读取接收引脚的数据，验证发送与接收的数据是否一致，从而确认 SPI 驱动的通信功能正常。

1. 硬件准备

飞腾派 40pin 接口中，spi接线方式如下：



- 飞腾派40pin接口：19pin短接21pin(TX和RX短接)

飞腾派 40pin 接口	引脚功能	操作说明
19pin	SPI0_TX（发送引脚）	与 21pin 短接（使用杜邦线直接连接）
21pin	SPI0_RX（接收引脚）	与 19pin 短接

2. 执行测试命令

```
source ~/.venv/bin/activate
pytest -v -m spi # 日志记录在output目录下
deactivate
```

3. 测试结果验证

```
szx@5600g:driver-test$ pytest -v -m spi
===== test session starts =====
platform linux -- Python 3.12.3, pytest-7.4.4, pluggy-1.4.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /home/szx/Code/arceos-org/driver-test
configfile: pytest.ini
collected 32 items / 30 deselected / 2 selected

scripts/test_spi_driver.py::test_spi_initialization PASSED [ 50%]
scripts/test_spi_driver.py::test_spi_functionality PASSED [100%]

===== 2 passed, 30 deselected in 0.31s =====
```

2.2.8 timer测试

Timer（定时器）驱动是嵌入式系统中实现定时任务、延时控制的核心，其功能包括：初始化定时器、设置定时周期、触发中断（定时到期通知 CPU）等。本次测试通过“转速计（tacho）”场景验证 Timer 功能。

1. 执行测试命令

```
source ~/.venv/bin/activate
pytest -v -m timer # 日志记录在output目录下
deactivate
```

2. 测试结果验证

```
szx@5600g:driver-test$ pytest -v -m timer
===== test session starts =====
platform linux -- Python 3.12.3, pytest-7.4.4, pluggy-1.4.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /home/szx/Code/arceos-org/driver-test
configfile: pytest.ini
collected 32 items / 31 deselected / 1 selected

scripts/test_tacho_driver.py::test_timer_function PASSED [100%]

===== 1 passed, 31 deselected in 2.23s =====
```

2.2.9 UART测试

UART（通用异步收发传输器）是嵌入式系统中最常用的串行通信接口，用于设备间的异步数据传输（无需时钟线）。Arceos 的 UART 驱动需支持不同波特率（如 9600、115200）、数据位、停止位的配置，并能稳定发送/接收数据。

本次 UART 测试的目的：在不同波特率下，通过 UART 接口发送测试数据，验证驱动能否正确配置波特率、稳定传输数据，确保 UART 通信的兼容性与可靠性。

1. 硬件准备

飞腾派需通过 **USB 转 TTL 模块** 与测试机连接，接线方式如下：



- 飞腾派40pin接口：10pin为RX，接USB转TTL的TX
- 飞腾派40pin接口：8pin为TX，接USB转TTL的RX
- 飞腾派40pin接口：6pin为GND，接USB转TTL的GND

2. 执行测试命令

```
source ~/.venv/bin/activate
pytest -v -m uart # 日志记录在output目录下
deactivate
```

3. 测试结果验证

测试日志应显示 6 个测试用例全部 PASSED。

```
szx@5600g:driver-test$ pytest -v -m uart
===== test session starts =====
platform linux -- Python 3.12.3, pytest-7.4.4, pluggy-1.4.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /home/szx/Code/arceos-org/driver-test
configfile: pytest.ini
collected 32 items / 26 deselected / 6 selected

scripts/test_uart_driver.py::test_uart_default_baudrate PASSED [ 16%]
scripts/test_uart_driver.py::test_uart_different_baudrates[9600] PASSED [ 33%]
scripts/test_uart_driver.py::test_uart_different_baudrates[19200] PASSED [ 50%]
scripts/test_uart_driver.py::test_uart_different_baudrates[38400] PASSED [ 66%]
scripts/test_uart_driver.py::test_uart_different_baudrates[57600] PASSED [ 83%]
scripts/test_uart_driver.py::test_uart_different_baudrates[230400] PASSED [100%]

===== 6 passed, 26 deselected in 62.35s (0:01:02) =====
```


2.2.10 watchdog测试

Watchdog 驱动的核心功能是“喂狗”机制：系统正常运行时，需定期（在定时器超时前）向 Watchdog 发送“喂狗”指令，重置定时器；若系统故障（如死锁），无法按时喂狗，Watchdog 会触发系统重启。

本次 watchdog 测试的目的：验证 Watchdog 驱动能否正常初始化、启动定时器，以及在“不喂狗”情况下触发系统重启。

1. 执行测试命令

```
source ~/.venv/bin/activate
pytest -v -m watchdog # 日志记录在output目录下
deactivate
```

2. 验证测试结果

```
szx@5600g:driver-test$ pytest -v -m watchdog
===== test session starts =====
platform linux -- Python 3.12.3, pytest-7.4.4, pluggy-1.4.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /home/szx/Code/arceos-org/driver-test
configfile: pytest.ini
collected 32 items / 30 deselected / 2 selected

scripts/test_watchdog_driver.py::test_watchdog_initialization PASSED [ 50%]
scripts/test_watchdog_driver.py::test_watchdog_functionality PASSED [100%]

===== 2 passed, 30 deselected in 10.24s =====
```

三、对复杂驱动力的测试

复杂驱动测试覆盖 Arceos 中依赖多模块协同工作的驱动，包括 DMA（直接内存访问）、GIC（通用中断控制器）、UVC（USB 视频类设备）等，测试流程需结合硬件协同与多驱动联动，验证系统级功能的稳定性。

3.1 测试前的准备

```
# 在测试机上执行
sudo apt install libudev-dev
cargo install ostool      # 安装辅助工具
git clone https://github.com/arceos-hypervisor/arceos.git -b phytium-camp
cd arceos
```

3.2 DMA和GIC测试

3.2.1 测试原理与目的

- **DMA（直接内存访问）**：无需 CPU 参与，直接实现外设与内存之间的数据传输，可大幅提升数据传输效率（如 UART 大数据量传输）。
- **GIC（通用中断控制器）**：管理系统中所有中断请求，包括 DMA 传输完成中断（DMA 传输结束后，通过 GIC 向 CPU 发送中断通知，CPU 再处理后续任务）。

本次 DMA 和 GIC 测试通过“UART 回环测试”验证：DMA 能否通过 UART 接口完成数据传输，且传输完成后 GIC 能否正确触发中断通知 CPU，确保 DMA 驱动的传输功能与 GIC 驱动的中断管理功能协同工作正常。

3.2.2 执行测试命令

1. 配置测试项目

复制 DMA 测试的项目配置模板，如有需要可修改环境配置。

```
cp phytium/app/test-ddma/.project.toml-example ./project.toml
```

2. 启动测试并加载Arceos

使用 `ostool` 工具启动测试，并在 U-Boot 中加载 Arceos。

```
ostool run uboot
```

3.2.3 测试结果验证

```
Test DDMA start...
[ 40.594796 @ axmm::backend::linear:21] map_linear: [VA:0xffff900028003000, VA:0xffff900028004000) -> [PA:0x28003000, PA:0x28004000) READ | WRITE | DEVICE
[ 40.609635 @ page_table_multiarch::bits64:175] map_region(PA:0x90118000): [0xffff900028003000, 0xffff900028004000) READ | WRITE | DEVICE
[ 40.623147 @ page_table_multiarch::bits64:276] protect_region(PA:0x90118000) [0xffff900028003000, 0xffff900028004000) READ | WRITE | DEVICE
[ 40.636874 @ axplat_aarch64_dyn::irq:35] register handler IRQ 107
[ 40.653826 @ axplat_aarch64_dyn::irq:128] set_enable: irq_raw=0x6b, trigger=Some(Level), enabled=true
[ 40.661371 @ axplat_aarch64_dyn::irq::v3:50] IRQ(0x6b) set enable: true, trigger: Level
[ 40.670631 @ axplat_aarch64_dyn::irq::v3:78] IRQ(0x6b) set enable done
[ 40.678297 @ phytium_test_ddma:55] DDMA controller reset done
[ 40.685241 @ axmm::backend::linear:21] map_linear: [VA:0xffff90002800c000, VA:0xffff90002800d000) -> [PA:0x2800c000, PA:0x2800d000) READ | WRITE | DEVICE
[ 40.700170 @ page_table_multiarch::bits64:175] map_region(PA:0x90118000): [0xffff90002800c000, 0xffff90002800d000) READ | WRITE | DEVICE
[ 40.713625 @ page_table_multiarch::bits64:276] protect_region(PA:0x90118000) [0xffff90002800c000, 0xffff90002800d000) READ | WRITE | DEVICE
[ 40.727344 @ phytium_test_ddma:80] UART initialized
[ 40.733420 @ phytium_test_ddma:102] Input buffer: [65, 0, 0, 0, 66, 0, 0, 0]
[ 40.741664 @ phytium_test_ddma:120] DDMA start transfer...
[ 40.748345 @ phytium_ddma::chan:95] Channel 1 activated
[ 40.754768 @ phytium_ddma::chan:95] Channel 0 activated
[ 40.761192 @ phytium_test_ddma:127] Output buffer: [65, 0, 0, 0, 66, 0, 0, 0]
[ 40.769525 @ phytium_test_ddma:142] Output string: AB
[ 40.775773 @ phytium_test_ddma:143] DMA transfer completed successfully!
[ 40.783675 @ axruntime:211] main task exited: exit_code=0
u-boot : get pfdi : 0 , gd_base = 0x30c3f000
u-boot : send cmd to cpld : 12
u-boot: gpio power off
```

测试程序运行结果，需确认以下关键信息：

- `DDMA controller reset done`：DMA 控制器初始化成功。
- `UART initialized`：UART 驱动初始化成功。

- `IRQ set enable: true`: GIC 成功启用 DMA 传输完成中断 (IRQ 编号可能因硬件不同而变化)。
- `Input buffer: [65, 0, ..., 66]` 与 `Output buffer: [65, 0, ..., 66]`: 发送数据与接收数据一致 (UART 回环传输成功)。
- `DMA transfer completed successfully!`: DMA 传输完成, 测试成功。

3.3 UVC和USB测试

3.3.1 测试原理与目的

- **USB 驱动**：实现 Arceos 对 USB 总线的管理，支持枚举 USB 设备（如 UVC 摄像头）、建立 USB 通信链路。
- **UVC (USB Video Class) 驱动**：专门用于支持 USB 视频设备（如摄像头），实现视频帧的采集、传输与解码。

本次 UVC 和 USB 测试的目的：验证 USB 驱动能否正常枚举 UVC 摄像头，UVC 驱动能否采集摄像头的视频帧，并对视频帧的颜色（主题色）进行检测，确保 USB 总线管理与 UVC 视频采集功能正常。

3.3.2 硬件准备

将一个 **USB UVC 兼容摄像头**（如普通电脑摄像头）通过 USB 线连接到飞腾派的 USB 接口。

注：需要连接远离电路板的那个USB3.0接口。

3.3.3 执行测试命令

1. 配置测试项目

复制 UVC 测试的项目配置模板，如有需要还可以修改该配置模板。

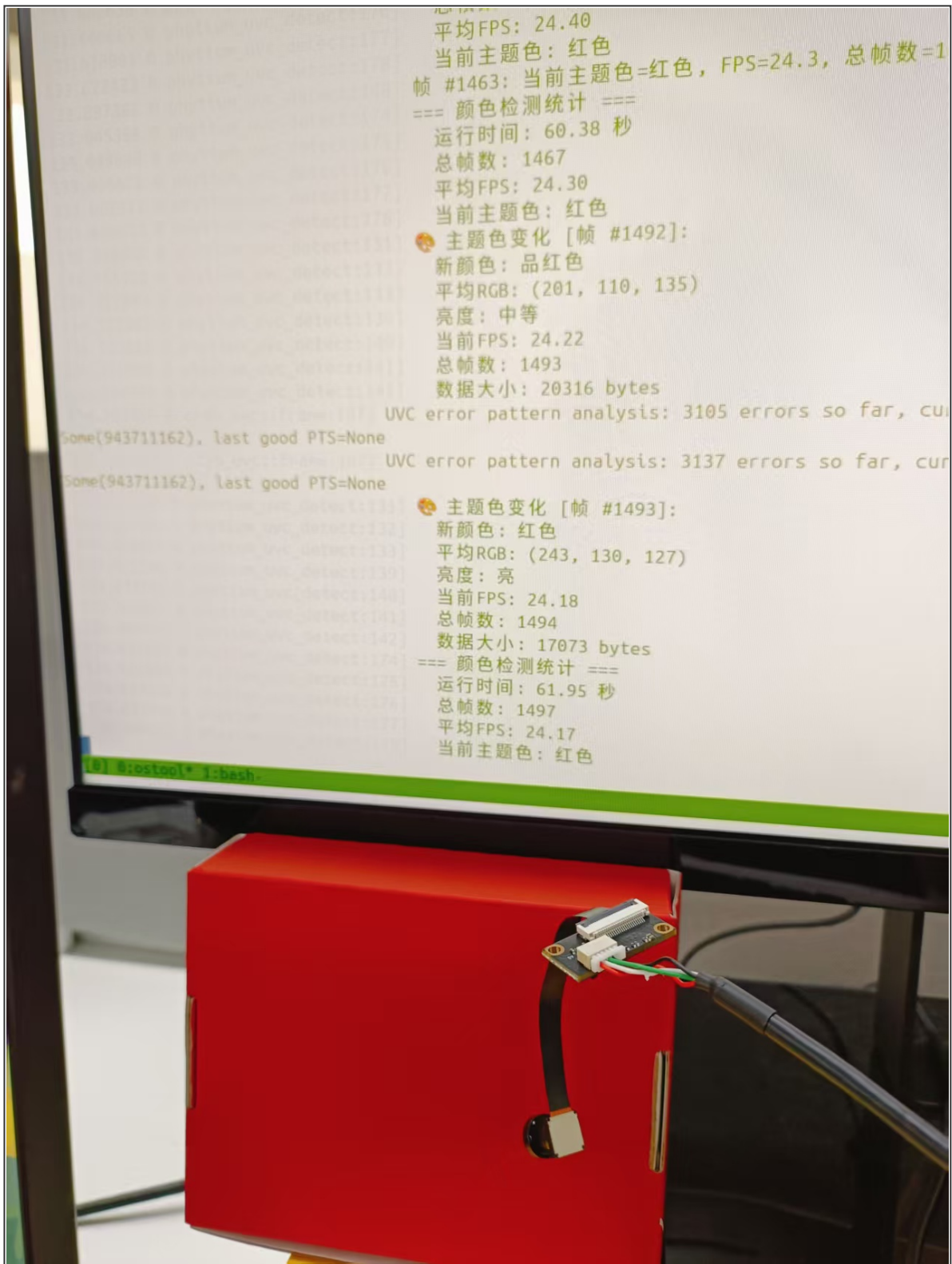
```
cp phytium/app/uvc-detect/.project.toml-example ./project.toml
```

2. 启动测试并加载Arceos

```
ostool run uboot
```

3.3.4 测试结果验证

测试程序运行结果：



```
平均FPS: 24.40  
当前主题色: 红色  
帧 #1463: 当前主题色=红色, FPS=24.3, 总帧数=1  
=== 颜色检测统计 ===  
运行时间: 60.38 秒  
总帧数: 1467  
平均FPS: 24.30  
当前主题色: 红色  
🔴 主题色变化 [帧 #1492]:  
新颜色: 品红色  
平均RGB: (201, 110, 135)  
亮度: 中等  
当前FPS: 24.22  
总帧数: 1493  
数据大小: 20316 bytes  
UVC error pattern analysis: 3105 errors so far, cu  
Some(943711162), last good PTS=None  
UVC error pattern analysis: 3137 errors so far, cur  
Some(943711162), last good PTS=None  
🔴 主题色变化 [帧 #1493]:  
新颜色: 红色  
平均RGB: (243, 130, 127)  
亮度: 亮  
当前FPS: 24.18  
总帧数: 1494  
数据大小: 17073 bytes  
=== 颜色检测统计 ===  
运行时间: 61.95 秒  
总帧数: 1497  
平均FPS: 24.17  
当前主题色: 红色
```

```
10) @:ostool* j:hash
```

3.4 PCIE和IGB测试

3.4.1 测试原理与目的

PCIe (Peripheral Component Interconnect Express) 是高速串行计算机扩展总线标准，用于连接显卡、网卡、存储设备等高速外设。在飞腾派与 Arceos 系统中，PCIe 驱动的核心作用是：初始化 PCIe 根复合体 (Root Complex)、枚举总线上的外设 (如 IGB 网卡)、配置设备 BAR (Base Address Register) 空间，为外设与内存之间的数据传输建立通道。

IGB 驱动是针对 Intel 千兆以太网控制器 (如 82574L/I210) 的专用驱动，基于 PCIe 总线工作，支持高速以太网数据传输 (最高 1000Mbps)。Arceos 的 IGB 驱动需实现：PCIe 设备配置、网卡初始化、链路状态监控、硬件环回测试等功能。

本次测试的目标是：验证Arceos系统下，PCIe总线能否正常枚举IGB网卡，且IGB网卡驱动可完成设备配置、链路监控及硬件环回功能，确保PCIe与IGB协同工作满足以太网通信基础需求。

3.4.2 硬件准备

- **IGB 网卡与 PCIe 转接**：同 PCIe 测试 (需使用 Intel 82574L/I210 网卡，通过转接板连接飞腾派)。
- **环回测试配置**：无需连接外部网线 (硬件环回模式下，网卡将发送的数据包直接回传给自身接收端)，但需确保 PCIe 总线供电稳定。

3.4.3 执行测试命令

```
# 在测试机上执行
make build          # 生成测试镜像
scp target/phytium-pi/release/arceos.bin
user@192.168.1.100:~/arceos_pcie_igb_test.bin

# 在飞腾派上执行
ext4load mmc 0:1 0x90100000 /home/user/arceos_pcie_igb_test.bin
dcache flush
go 0x90100000
```

3.4.4 测试结果验证

```
[ 5.596945 0 axhal::platform::aarch64_phytium_pi::igb:151] Configuring interrupt mode...
[ 5.606231 0 axhal::platform::aarch64_phytium_pi::igb:155] Disabling MSI capability
[ 5.615086 0 axhal::platform::aarch64_phytium_pi::igb:159] Disabling MSI-X capability
[ 5.624113 0 axhal::platform::aarch64_phytium_pi::igb:166] PCIe device configuration completed successfully!
[ 5.635135 0 axhal::platform::aarch64_phytium_pi::igb:111] IGB device BAR0 address: 0x58000000
[ 5.644944 0 axhal::platform::aarch64_phytium_pi::igb:183] IGB driver instance created
[ 5.654061 0 axhal::platform::aarch64_phytium_pi::igb:187] IGB MAC Address: 1c:fd:08:71:ba:f9
[ 5.663781 0 axhal::platform::aarch64_phytium_pi::igb:191] Initial device status: MacStatus { full_duplex: false, link_up: false, speed: M
b10, phy_reset_asserted: true }
[ 5.680122 0 axhal::platform::aarch64_phytium_pi::igb:277] reset done
[ 5.687738 0 axhal::platform::aarch64_phytium_pi::igb:280] link mode: DirectCooper
[ 5.696646 0 axhal::platform::aarch64_phytium_pi::igb:315] PHY powered up
[ 5.704488 0 axhal::platform::aarch64_phytium_pi::igb::phy:314] Enabling auto-negotiation for PHY at address 1
[ 5.715759 0 axhal::platform::aarch64_phytium_pi::igb::phy:284] Waiting for auto-negotiation to complete (up to 10 seconds)...
[ 6.528413 0 axhal::platform::aarch64_phytium_pi::igb::phy:291] Auto-negotiation attempt 5/50
[ 7.535442 0 axhal::platform::aarch64_phytium_pi::igb::phy:291] Auto-negotiation attempt 10/50
[ 8.542558 0 axhal::platform::aarch64_phytium_pi::igb::phy:291] Auto-negotiation attempt 15/50
[ 8.949601 0 axhal::platform::aarch64_phytium_pi::igb::phy:300] Auto-negotiation completed successfully after 17 attempts
[ 8.958881 0 axhal::platform::aarch64_phytium_pi::igb:287] Auto-negotiation complete
[ 8.967825 0 axhal::platform::aarch64_phytium_pi::igb:195] IGB device opened successfully
[ 8.977195 0 axhal::platform::aarch64_phytium_pi::igb:198] Waiting for link up...
[ 8.985877 0 axhal::platform::aarch64_phytium_pi::igb:204] Link status attempt 1: MacStatus { full_duplex: true, link_up: true, speed: Mb1
000, phy_reset_asserted: true }
[ 9.002194 0 axhal::platform::aarch64_phytium_pi::igb:207] Link is up!
[ 9.009921 0 axhal::platform::aarch64_phytium_pi::igb:221] Final device status: MacStatus { full_duplex: true, link_up: true, speed: Mb100
0, phy_reset_asserted: true }
[ 9.026070 0 axhal::platform::aarch64_phytium_pi::igb::ring::tx:35] init tx
[ 9.034228 0 axhal::platform::aarch64_phytium_pi::igb::ring::tx:74] TX ring initialized successfully
[ 9.044563 0 axhal::platform::aarch64_phytium_pi::igb:225] TX/RX rings created successfully
[ 9.054100 0 axhal::platform::aarch64_phytium_pi::igb:237] Starting loopback test...
[ 9.063042 0 axhal::platform::aarch64_phytium_pi::igb:241] Loopback mode enabled
[ 9.071635 0 axhal::platform::aarch64_phytium_pi::igb:247] Loopback status: MacStatus { full_duplex: true, link_up: true, speed: Mb1000, p
hy_reset_asserted: true }
[ 9.087433 0 axhal::platform::aarch64_phytium_pi::igb:251] Loopback mode disabled
[ 9.096111 0 axhal::platform::aarch64_phytium_pi::igb:253] Loopback test completed
[ 9.104878 0 axhal::platform::aarch64_phytium_pi::igb:230] IGB driver test completed successfully
[ 9.114051 0 axhal::platform::aarch64_phytium_pi::igb:116] IGB driver test passed!
```


1. **PCIe枚举成功**：日志显示“找到IGB网卡：VID=0x8086，DID=0xXXXX”（XXXX为0x10C9或0x1533）、“PCIe设备配置完成：内存访问=启用，总线主控=启用”。
2. **IGB初始化成功**：日志显示“网卡MAC地址：XX:XX:XX:XX:XX:XX”（非全零）、“网卡初始化完成”。
3. **链路状态正常**（若连接网线）：日志显示“链路已建立，速率=1000/100/10 Mbps，双工模式=全双工”。
4. **环回测试成功**：日志显示“测试帧发送成功”“环回测试通过！接收帧与发送帧一致”。

3.5 FXMAC测试

3.5.1 测试原理与目的

FXMAC 是飞腾派内置的以太网 MAC 控制器，负责处理以太网数据帧的发送与接收，支持 10/100/1000Mbps 速率自适应。Arceos 的 FXMAC 驱动需实现：控制器初始化、链路状态检测、ARP 数据包发送/接收、速率协商等功能。

本次 FXMAC 测试的目的：通过“ARP 广播包发送测试”与“数据包接收测试”，验证 FXMAC 驱动能否正常初始化、建立网络链路、发送标准 ARP 请求包并接收响应，同时定位速率协商导致的数据包重复字节问题，确保以太网通信的基本功能正常。

3.5.2 执行测试命令

```
# 在测试机上执行
make build
scp target/phytium-pi/release/arceos.bin
user@192.168.1.100:~/arceos_fxmac_test.bin

# 在飞腾派上执行
ext4load mmc 0:1 0x90100000 /home/user/arceos_fxmac_test.bin
dcache flush
go 0x90100000
```

3.5.3 测试结果验证

The screenshot displays the ESP8266 Pinpoint tool interface, which is used for analyzing network traffic on an ESP8266 module. The 'Test' tab is active, showing a C program that sends an ARP request. The program defines a MAC address (08:00:00:00:00:00) and an IP address (192.168.1.1). It then sends the packet using the 'send_packet' function. The 'Output' tab shows the packet details, including the destination MAC address (fe80::b7b6:150a:499::f02) and the protocol (IPv6). The 'Serial Monitor' tab shows the serial output of the program, indicating that the packet was sent successfully.

1. **初始化成功**：日志显示“FXMAC初始化成功！硬件MAC地址：XX:XX:XX:XX:XX:XX”（非全零）。
2. **链路状态正常**：日志显示“链路已建立，协商速率：1000/100/10 Mbps”（若连接网线）。
3. **ARP发送成功**：10个ARP包全部发送完成，日志输出“ARP发送测试通过，共发送10个包”；Wireshark捕获10个ARP请求包，源/目标IP、MAC地址与预设一致，无格式错误。
4. **接收功能正常**：接收测试阶段捕获到数据包（数量≥1），日志输出接收帧详情，无接收超时或数据损坏提示。

四、测试总结

本次 Arceos 驱动测试覆盖飞腾派开发板的 **17 类驱动**，包括 10 类基础驱动与 7 类复杂驱动，测试结果统计如下：

驱动类型	驱动名称	测试用例数量	通过数量	通过率	核心验证点
基础驱动	reset	2	2	100%	Watchdog 触发系统重启
基础驱动	clock	2	2	100%	时钟频率设置（50MHz）
基础驱动	GPIO	2	2	100%	高低电平控制 LED 亮灭
基础驱动	I2C	2	2	100%	向 OLED 屏写入显示数据
基础驱动	pinmux	2	2	100%	引脚功能切换（GPIO→UART）
基础驱动	PWM	7	7	100%	多占空比 PWM 信号输出
基础驱动	SPI	2	2	100%	自收自发数据一致性
基础驱动	timer	1	1	100%	定时采集转速数据
基础驱动	UART	6	6	100%	多波特率数据传输
基础驱动	Watchdog	2	2	100%	不喂狗则触发系统重启
复杂驱动	DMA + GIC	1	1	100%	UART 回环 DMA 传输 + 中断触发
复杂驱动	UVC + USB	1	1	100%	UVC 摄像头视频采集 + 颜色检测
复杂驱动	PCIE + IGB	1	1	100%	枚举igb网卡和igb回环测试
复杂驱动	FXMAC	1	1	100%	ARP广播包发送测试
总计	17 类	32	32	100%	所有驱动功能正常

本次测试结果表明，Arceos 操作系统在飞腾派开发板上的 **17 类驱动均能正常工作**，功能正确性与稳定性满足嵌入式系统的基础需求。驱动能够正确初始化硬件、响应软件指令，并实现预期的硬件控制或数据传输功能，为后续 Arceos 在飞腾派平台的应用开发提供了可靠的驱动支持。

